

Algoritmo Differential Evolution (DE) aplicado al QAP

Explicación extendida, fundamentos teóricos y procedimiento con *random keys*

Resumen

Este documento describe de manera detallada el algoritmo Differential Evolution (DE) y su aplicación al Quadratic Assignment Problem (QAP) mediante codificación *random keys*. Además de la adaptación práctica, se presentan los fundamentos teóricos de DE: su naturaleza poblacional, la idea central de la mutación diferencial como auto-escala basada en diferencias entre individuos, las variantes habituales en la notación DE/x/y/z, los mecanismos de cruce y la selección greedy. Se incluyen pseudocódigo y diagramas TikZ para consolidar una comprensión completa.

Índice

1. Marco teórico: optimización poblacional en continuo y lugar de DE	2
1.1. Algoritmos poblacionales y búsqueda directa	2
1.2. Qué hace especial a DE frente a otros enfoques	2
1.3. Notación estándar: DE/x/y/z	2
1.4. Componentes de DE (visión general)	2
1.5. Parámetros fundamentales: NP , F , CR	2
1.6. Selección greedy: por qué es crucial	3
1.7. Cruce binomial vs exponencial (idea)	3
2. Contexto: QAP y <i>random keys</i>	3
2.1. QAP (recordatorio breve)	3
2.2. Codificación <i>random keys</i>	3
3. Fundamentos del algoritmo DE	3
3.1. Qué hace DE y por qué se llama “diferencial”	3
3.2. Estructura general	4
3.3. Estrategia DE/rand/1/bin (la más típica)	4
3.4. Interpretación geométrica (muy útil para defensa)	4
4. Aplicación de DE a QAP: procedimiento completo	4
4.1. Representación y acotación	4
4.2. Evaluación	4
4.3. Pseudocódigo detallado (DE + QAP + <i>random keys</i>)	5
4.4. Qué controla cada parámetro (clave para defender)	5
5. Diagrama de flujo (TikZ): DE aplicado a QAP	6
6. Preguntas típicas de defensa (y respuestas)	7
7. Conclusión	7

1. Marco teórico: optimización poblacional en continuo y lugar de DE

1.1. Algoritmos poblacionales y búsqueda directa

DE pertenece a la familia de algoritmos poblacionales para optimización en espacios continuos. Se caracterizan por:

- Mantener un conjunto de soluciones y usar su diversidad como “información” para explorar.
- No requerir derivadas (métodos de búsqueda directa).
- Poder afrontar funciones no convexas, discontinuas o ruidosas (en general).

1.2. Qué hace especial a DE frente a otros enfoques

La característica distintiva de DE es que la variación principal se construye a partir de diferencias entre soluciones:

$$x_{r2} - x_{r3}.$$

Esa diferencia codifica (1) una dirección y (2) una escala, determinadas por la propia población. Por eso se dice que DE “se auto-adapta” en parte a la geometría del problema a través de la diversidad presente.

1.3. Notación estándar: DE/x/y/z

En la literatura, DE suele describirse como DE/x/y/z:

- **x**: vector base (rand, best, current-to-best, etc.).
- **y**: número de vectores diferencia usados (1, 2, ...).
- **z**: tipo de cruce (bin = binomial, exp = exponencial).

Ejemplos:

- DE/rand/1/bin (muy clásico): base aleatoria, una diferencia, cruce binomial.
- DE/best/1/bin: base el mejor individuo actual, una diferencia.
- DE/current-to-best/1/bin: empuja el target hacia el mejor (más explotación).

1.4. Componentes de DE (visión general)

DE se entiende por tres operaciones sobre cada individuo objetivo x_i :

1. **Mutación diferencial** para crear un mutante v .
2. **Cruce** (binomial/exponencial) para crear un trial u .
3. **Selección greedy** entre u y x_i .

1.5. Parámetros fundamentales: NP , F , CR

NP (tamaño de población). Controla diversidad y coste por iteración.

F (factor de escala). Escala la diferencia: si F es grande, la exploración es más agresiva; si es pequeño, los pasos son más finos.

CR (crossover rate). Controla cuánto del mutante entra en el trial. CR alto favorece trial más parecido al mutante.

1.6. Selección greedy: por qué es crucial

La selección greedy (si el trial mejora, reemplaza) es uno de los rasgos más característicos de DE:

- Asegura que cada “posición” de la población no empeora al aceptar trial.
- Introduce una presión selectiva directa, a nivel de individuo.
- Simplifica el análisis del flujo: trial compite sólo con su target.

1.7. Cruce binomial vs exponencial (idea)

Binomial (bin). Decide por componente con probabilidad CR.

Exponencial (exp). Copia un segmento contiguo de componentes desde el mutante; útil en algunos problemas donde la correlación entre variables es local.

En tu implementación se usa binomial (habitual y suficiente para TFG).

2. Contexto: QAP y random keys

2.1. QAP (recordatorio breve)

El coste para una permutación $p \in \mathcal{S}_n$ es:

$$C(p) = \sum_{i=0}^{n-1} \sum_{j=0}^{n-1} F_{ij} D_{p(i)p(j)} \quad (\text{minimización}).$$

2.2. Codificación random keys

Se representa cada individuo como $x \in \mathbb{R}^n$ (habitualmente acotado en $[0, 1]^n$) y se decodifica:

$$p = \text{argsort}(x).$$

DE actúa sobre x ; el QAP se evalúa sobre p .

3. Fundamentos del algoritmo DE

3.1. Qué hace DE y por qué se llama “diferencial”

DE es un método poblacional que genera, para cada individuo objetivo (*target*), un individuo candidato (*trial*) mediante:

- **Mutación diferencial:** combina un vector base con una diferencia escalada entre dos vectores.
- **Cruce:** mezcla target y mutante para producir el trial.
- **Selección greedy:** el trial reemplaza al target si mejora (en minimización, si su coste es menor o igual).

Se denomina “diferencial” porque el paso de búsqueda usa un vector diferencia ($x_{r2} - x_{r3}$) que codifica dirección y magnitud a partir de la diversidad de la población.

3.2. Estructura general

- Población $P = \{x^{(1)}, \dots, x^{(NP)}\}$ de tamaño NP .
- Parámetros principales:
 - F (factor de escala) controla la longitud del salto.
 - CR (crossover rate) controla cuánto del mutante entra en el trial.

3.3. Estrategia DE/rand/1/bin (la más típica)

Para cada individuo objetivo x_i :

1. Elegir índices distintos $r1, r2, r3 \neq i$.
2. Mutante:

$$v = x_{r1} + F \cdot (x_{r2} - x_{r3}).$$

3. Cruce binomial (componente a componente):

$$u_j = \begin{cases} v_j, & \text{si } \text{rand}() < CR \text{ o } j = j' \\ x_{i,j}, & \text{en otro caso,} \end{cases}$$

donde j fuerza que al menos una componente provenga de v .

4. Selección greedy: si $f(u) \leq f(x_i)$, entonces $x_i \leftarrow u$.

3.4. Interpretación geométrica (muy útil para defensa)

El vector $(x_{r2} - x_{r3})$ es una “dirección” entre dos individuos. Multiplicarlo por F ajusta la amplitud. Sumárselo a x_{r1} crea un punto nuevo que hereda estructura global de la población (auto-adaptación basada en diversidad).

4. Aplicación de DE a QAP: procedimiento completo

4.1. Representación y acotación

Se usa $x \in [0, 1]^n$ para:

- controlar magnitudes,
- evitar explosión numérica,
- mantener estabilidad.

Tras mutación/cruce, se aplica **recorte** (clamping) componente a componente:

$$u_j \leftarrow \min(1, \max(0, u_j)).$$

Esto mantiene el dominio consistente.

4.2. Evaluación

Para evaluar un individuo x :

$$p = \text{argsort}(x), \quad \text{fitness}(x) = C(p).$$

En implementación, puede vectorizarse el coste reordenando D como $D_p = D[p, :][:, p]$ y computando $\sum F \odot D_p$.

4.3. Pseudocódigo detallado (DE + QAP + random keys)

```
procedure DE_QAP(Fmat, Dmat, NP, F, CR, max_evals)
  # 1) Inicialización
  P <- {NP vectores x ~ Uniforme([0,1]^n)}
  evaluar fitness de cada x: C(argsort(x))
  evals <- NP

  while evals < max_evals:
    for i in 1..NP:
      # 2) Selección de índices r1,r2,r3 distintos y != i
      r1,r2,r3 <- sample_distintos({1..NP}\{i})

      # 3) Mutación diferencial (DE/rand/1)
      v <- P[r1] + F * (P[r2] - P[r3])

      # 4) Cruce binomial target/mutante
      u <- empty vector
      j* <- random index in {1..n}
      for j in 1..n:
        if rand() < CR or j == j*:
          u[j] <- v[j]
        else:
          u[j] <- P[i][j]

      # 5) Recorte a [0,1]
      u <- clip(u, 0, 1)

      # 6) Evaluación y selección greedy
      fu <- C(argsort(u))
      evals <- evals + 1

      if fu <= fitness(P[i]):
        P[i] <- u
        fitness(P[i]) <- fu

    return mejor individuo de P
end procedure
```

4.4. Qué controla cada parámetro (clave para defender)

NP (tamaño de población). Mayor *NP* implica más diversidad pero más evaluaciones por generación.

F (factor de escala). Controla el tamaño del salto:

- *F* pequeño: pasos cortos, explotación.
- *F* grande: exploración más agresiva (pero puede desestabilizar).

CR (probabilidad de cruce). Controla cuántas componentes vienen del mutante:

- *CR* alto: trial se parece al mutante.

- CR bajo: trial se parece al target.

Recorte a $[0, 1]$. Mantiene dominio estable. En random keys, incluso si hay empates, **argsort** es determinista (típicamente por índice), así que el proceso sigue siendo reproducible.

5. Diagrama de flujo (TikZ): DE aplicado a QAP

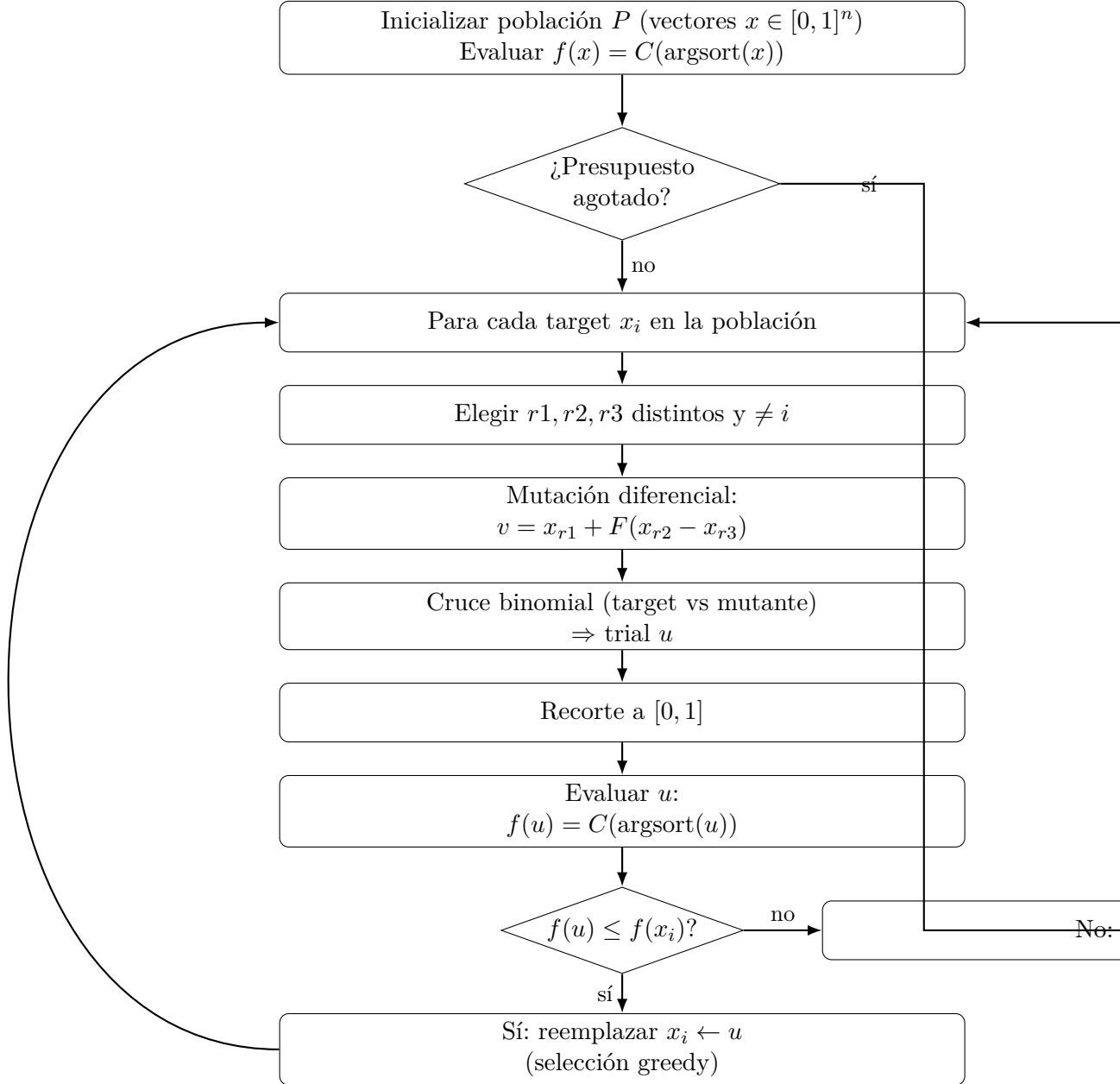


Figura 1: Flujo conceptual de DE/rand/1/bin aplicado al QAP con codificación *random keys*.

6. Preguntas típicas de defensa (y respuestas)

(1) ¿Por qué DE funciona sin gradientes?

Porque explora mediante diferencias entre soluciones existentes (información poblacional). No requiere derivadas ni continuidad estricta del objetivo.

(2) ¿Qué optimiza DE realmente con random keys?

Optimiza la función compuesta:

$$f(x) = C(\text{argsort}(x)).$$

El espacio continuo sirve para generar rankings (permutaciones) de forma indirecta.

(3) ¿Qué garantiza la selección greedy?

En minimización, garantiza que el fitness de cada posición de la población no empeora cuando se reemplaza (monotonía local).

(4) ¿Qué riesgos hay?

- Si F o CR son extremos, puede haber estancamiento o saltos demasiado agresivos.
- El recorte puede crear componentes iguales, afectando el ranking; aun así el proceso sigue siendo determinista.

7. Conclusión

DE aplicado al QAP mediante *random keys* conserva el núcleo del algoritmo (mutación diferencial, cruce binomial y selección greedy) y delega la naturaleza combinatoria del QAP a una decodificación determinista mediante **argsort**. Entender el rol de F , CR y NP , así como la interpretación geométrica de la mutación diferencial, permite explicar de forma sólida el comportamiento del algoritmo durante la defensa.